Д. Серікбаев атындағы Шығыс Қазақстан техникалық университеті

«DevOps құралдары» пәні

ЗЕРТХАНАЛЫҚ ЖҰМЫС № 2, 2-бөлім

Docker контейнерін іске қосу және тексеру

Орындаған: Есімханова Аружан Түсіпханқызы

Тобы: 25-ИСТК-2о

Оқу жылы: 2025-2026

Өскемен - 2026

МАЗМҰНЫ

1 Жұмыстың мақсаты 3

2 Теориялық мәліметтер 3

3 Жұмыстың орындалу барысы 4

4 Жұмыс нәтижелері 11

5 Талдау 11

6 Қорытынды 11

7 Пайдаланылған әдебиеттер тізімі 12

1 Жұмыстың мақсаты

DemoApi контейнерінің күйін, HTTP жауаптарын, журналдарын және орындалу ортасын тексеріп, Swagger 404/200 жауаптарының себебін түсіндіру.

2 Теориялық мәліметтер

Бейне - контейнерді іске қосуға арналған файлдар жиынтығы. Контейнер ішінде жеке процесс орындалады; оның журналы stdout арқылы жиналады. Healthcheck қызметтің күйін тексереді. ASP.NET Core ортасы қолданба маршруттарының тіркелуіне әсер етеді.

3 Жұмыстың орындалу барысы

Тапсырма: №04, 4-апта, 2-зертханалық жұмыстың 2-бөлімі.
Тәжірибе күні: 22.10.2026.
Есеп күні: 01.11.2026.

3.1 Жұмыс ортасы

1-кесте - Орындау деректері мен тексеру нәтижелері

Параметр	Мәні
Қолданба	DemoApi, ASP.NET Core / .NET 10
Docker Engine	29.8.0; Linux/amd64
Бейне	devops-labs-demo-api:lab02-1.1
Production контейнері	devops-labs-lab02-api; 127.0.0.1:8089 → 8080
Development контейнері	devops-labs-lab02-swagger; 127.0.0.1:8090 → 8080
Қойма	InMemory
Нұсқа	lab02-1.1

Есепте 22 қазанда сақталған контейнер, HTTP және тест нәтижелері қолданылды. Тәжірибеге сәйкес код runtime-checks.json файлымен және lab02-docker тегімен байланысты.

3.2 Соңғы Dockerfile

1-листинг - Код немесе сақталған команда шығысы

```
# syntax=docker/dockerfile:1

FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY src/DemoApi/DemoApi.csproj src/DemoApi/
RUN dotnet restore src/DemoApi/DemoApi.csproj
COPY src/ src/
RUN dotnet publish src/DemoApi/DemoApi.csproj -c Release -o /app --no-restore

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
RUN apt-get update \
    && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY --from=build /app .
ARG APP_VERSION=dev
```

```
ENV APP_VERSION=$APP_VERSION
USER $APP_UID
EXPOSE 8080
HEALTHCHECK --interval=10s --timeout=3s --start-period=15s --retries=3 \
    CMD curl -fsS http://localhost:8080/health/live || exit 1
ENTRYPOINT ["dotnet", "DemoApi.dll"]
```

build кезеңінде SDK арқылы тәуелділіктер қалпына келтіріліп, Release жинағы жарияланды. runtime кезеңіне /app ішіндегі жарияланған файлдар көшірілді. curl құралы healthcheck сұрауын орындайды. USER $APP_UID қолданбаны root құқықтарынсыз іске қосады.

3.3 .dockerignore

2-листинг - Код немесе сақталған команда шығысы

```
**/bin/
**/obj/
**/TestResults/
.git
.github
.vs
.vscode
.idea
tests/
monitoring/
scripts/
db/
docs/
reports/
evidence/
postman/
*.md
Dockerfile
compose*.yaml
compose*.yml
.env
.env.*
**/.env
**/.env.*
**/*.pfx
**/*.p12
```

bin, obj және TestResults каталогтары, Git тарихы, редактор параметрлері, тесттер, есептер және жергілікті құпия файлдар құрастыру контекстіне кірмейді. Dockerfile мен .dockerignore мазмұны lab02-docker нұсқасымен салыстырылды.

3.4 Контейнерді іске қосу және күйін тексеру

3-листинг - Код немесе сақталған команда шығысы

```
docker run -d --name devops-labs-lab02-api -p 127.0.0.1:8089:8080 -e Storage=InMemory
 devops-labs-demo-api:lab02-1.1
docker ps --filter name=devops-labs-lab02-api
docker inspect --format "{{.State.Health.Status}}" devops-labs-lab02-api
```

Контейнер іске қосылғаннан кейін healthcheck /health/live мекенжайын тексерді. Docker inspect нәтижесі бойынша процесс жұмыс істеп тұрды, Health.Status мәні healthy болды. Нақты күй health-production.json файлында сақталған.

4-листинг - Код немесе сақталған команда шығысы

```
{
  "Status": "running",
  "Running": true,
  "Health": {
    "Status": "healthy",
    "FailingStreak": 0,
    "Log": [
      {
        "ExitCode": 0,
        "Output": "Healthy"
      }
    ]
  }
}
```

Healthcheck аралығы - 10 секунд, timeout - 3 секунд, бастапқы күту кезеңі - 15 секунд, қайталау саны - 3. Хосттағы 8089 порты контейнердің 8080 портына бағытталған.

3.5 GET / және health маршруттары

5-листинг - Код немесе сақталған команда шығысы

```
curl.exe -i http://localhost:8089/
curl.exe -i http://localhost:8089/health/live
curl.exe -i http://localhost:8089/health/ready
```

6-листинг - Код немесе сақталған команда шығысы

```
GET / → HTTP 200
{
  "service": "DemoApi",
  "version": "lab02-1.1",
  "host": "e7ee83006c4d"
}
```

version өрісі құрастыру кезінде берілген lab02-1.1 мәнін көрсетті. host өрісіндегі e7ee83006c4d - контейнердің hostname мәні; ол Windows компьютерінің атауы емес.

2-кесте - Орындау деректері мен тексеру нәтижелері

Сұрау	HTTP коды	Жауап
GET /	200	service=DemoApi; version=lab02-1.1
GET /health/live	200	Healthy
GET /health/ready	200	Healthy

3.6 Logs: қолданба журналы

7-листинг - Код немесе сақталған команда шығысы

```
docker logs devops-labs-lab02-api
```

Журналдан InMemory қоймасы, контейнер ішіндегі 8080 порты, Production ортасы және /app жұмыс каталогы анықталды. Төменде api-logs.txt файлындағы алғашқы бес JSON жазбасының Message өрістері берілген.

8-листинг - Код немесе сақталған команда шығысы

```
Хранилище: в памяти
Now listening on: http://[::]:8080
Application started. Press Ctrl+C to shut down.
Hosting environment: Production
Content root path: /app
```

Контейнердегі стандартты шығыс Docker журналына жиналады. Docker Desktop ішіндегі Logs вкладкасы осы ағынды көрсетуге арналған. Бұл есепте сақталған docker logs шығысы пайдаланылды.

3.7 Exec: контейнер ішіндегі тексерулер

9-листинг - Код немесе сақталған команда шығысы

```
docker exec devops-labs-lab02-api id
docker exec devops-labs-lab02-api dotnet --list-sdks
docker exec devops-labs-lab02-api find /app -type f
```

10-листинг - Код немесе сақталған команда шығысы

```
uid=1654(app) gid=1654(app) groups=1654(app)
```

3-кесте - Орындау деректері мен тексеру нәтижелері

Тексеру	Нақты нәтиже
id	uid=1654(app); процесс root емес
dotnet --list-sdks	Бос шығыс: соңғы бейнеде SDK жоқ
find /app -type f	Жарияланған файлдар бар; .cs, .csproj және .slnx жоқ
Кіру нүктесі	dotnet DemoApi.dll

Exec тексерулері контейнердің орындалу ортасын көрсетті. SDK тек build кезеңінде қалды. /app ішіндегі файлдардың толық тізімі runtime-files.txt файлында, UID мен SDK тексерулері runtime-checks.json файлында сақталған.

3.8 Контейнер қайта іске қосылғанда деректердің өзгеруі

InMemory қоймасында «Lab02 restart evidence» жобасы құрылды. Одан кейін сол контейнер docker stop және docker start арқылы қайта іске қосылды. Контейнердің өзі сақталғанымен, қолданба процесінің жедел жады қайта құрылды.

11-листинг - Код немесе сақталған команда шығысы

```
docker stop devops-labs-lab02-api
docker start devops-labs-lab02-api
curl.exe http://localhost:8089/api/projects
```

4-кесте - Орындау деректері мен тексеру нәтижелері

Кезең	HTTP коды	Жобалар саны
Қайта іске қосуға дейін	200	1
Қайта іске қосқаннан кейін	200	0

Нақты жауаптар restart-before.json және restart-after.json файлдарында сақталған. InMemory деректері контейнердің файлдық жүйесінде емес, процесс жадында болғандықтан жоғалды. Тұрақты сақтау үшін дерекқор мен оның деректер томы қажет.

3.9 Swagger: Production ортасындағы 404 және Development ортасындағы 200

Алғашқы контейнерде ASPNETCORE_ENVIRONMENT айнымалысы берілмегендіктен, Production ортасы қолданылды. Program.cs ішінде OpenAPI және Swagger тек app.Environment.IsDevelopment() шарты орындалғанда тіркеледі. Сондықтан Production контейнеріндегі Swagger сұрауы 404 қайтарды.

12-листинг - Код немесе сақталған команда шығысы

```
GET http://localhost:8089/swagger/index.html
HTTP 404
Body: ""
```

Сол devops-labs-demo-api:lab02-1.1 бейнесінен Development ортасымен екінші контейнер іске қосылды. Chaos__Enabled=false параметрі оқу ақауларын өшірді. Бейне қайта құрастырылған жоқ.

13-листинг - Код немесе сақталған команда шығысы

```
docker run -d --name devops-labs-lab02-swagger -p 127.0.0.1:8090:8080 -e Storage=InMe
mory -e ASPNETCORE_ENVIRONMENT=Development -e Chaos__Enabled=false devops-labs-demo-a
pi:lab02-1.1
curl.exe -i http://localhost:8090/swagger/index.html
```

5-кесте - Орындау деректері мен тексеру нәтижелері

Орта	Мекенжай	HTTP коды
Production	localhost:8089/swagger/index.html	404
Development	localhost:8090/swagger/index.html	200
Development	localhost:8090/openapi/v1.json	200

404 жауабы бұл жағдайда сервистің тоқтағанын білдірмейді: GET / және health маршруттары 200 қайтарды. Айырмашылық - Swagger маршрутының ортаға байланысты тіркелуі. Екі жауап swagger-production.json және swagger-development.json файлдарында берілген.

3.10 Сақталған тест нәтижелері

6-кесте - Орындау деректері мен тексеру нәтижелері

Тексеру	Нәтиже
Контейнер мен API бойынша тексерулер	14/14 сәтті
HTTP сценарийлері	26 сұрау: 23 сценарий және 3 тазалау сұрауы
Postman CLI	24 сұрау; 73 тексеру; қате 0
.NET Release / InMemory	28/28; failed 0; skipped 0

Бұл нәтижелер 22.10.2026 күнгі lab02-1.1 нұсқасына жатады. Postman шығысы postman-console.txt файлында, .NET тест нәтижелері lab02-regression.trx файлында сақталған.

3.11 Жұмыс материалдары

Dockerfile; .dockerignore; Program.cs; контейнер тәжірибесінің журналдары.

4 Жұмыс нәтижелері

Контейнердің healthy күйі, GET / және health маршруттарының 200 жауабы расталды. UID 1654, SDK жоқ. Swagger Production ортасында 404, Development ортасында 200 қайтарды. InMemory жобалар саны stop/start кезінде 1-ден 0-ге өзгерді. Postman: 24 сұрау және 73 тексеру; .NET: 28/28.

Тест нәтижелері мен журналдар жұмыс материалдарында [1], іске қосу командалары нұсқаулықта [2] берілген.

5 Талдау

Қызметтің жұмыс істеуі мен Swagger қолжетімділігі бөлек тексерілді. Production ортасында Swagger тіркелмегендіктен 404 қайтты; Development ортасы сол бейнеде 200 берді. InMemory деректері процесс жадында сақталғандықтан қайта іске қосылғанда жоғалды.

6 Қорытынды

DemoApi Docker контейнерінде жұмыс істеді. Healthcheck, журналдар және контейнер ішіндегі командалар арқылы іске қосу ортасы тексерілді. 404/200 жауаптарының конфигурацияға тәуелділігі анықталды.

7 Пайдаланылған әдебиеттер тізімі

[1] DemoApi оқу репозиторийі. Нақты тексерулер мен сақталған дәлелдер. Электрондық ресурс. https://github.com/Aruzhaaan-06/devops-labs/blob/main/evidence/lab02/README.md. Қаралған күні: 01.11.2026.

[2] DemoApi оқу репозиторийі. Жұмыстың орындалу тәртібі. Электрондық ресурс. https://github.com/Aruzhaaan-06/devops-labs/blob/main/docs/lab02-part2.md. Қаралған күні: 01.11.2026.

DevOps пәнінің оқу материалдары: «Задание_04.pdf». 4-апта. 2-зертханалық жұмыс, 2-бөлім; тапсырманың тапсыру тізімі.

ГОСТ 7.32-2017. Ғылыми-зерттеу жұмысы туралы есеп. Құрылымы және рәсімдеу ережелері. 6-бөлім.